

Manual Definitivo: FSeleniumIA

Fundamentos de Java para QA Automation

Para dominar Selenium y automatizar pruebas como un Senior, primero debes comprender el idioma en el que estás hablando: Java. Este manual destila los conceptos clave de Java orientados exclusivamente a resolver los problemas del día a día en un framework de pruebas.

1. Programación Orientada a Objetos (POO)

Java es un lenguaje Orientado a Objetos. Todo gira en torno a Clases y Objetos.

Clases vs Objetos

- **Clase:** Es un "plano" o plantilla. Por ejemplo, la clase `PaginaLogin`. Una clase define qué datos (variables) y qué acciones (métodos) tiene.
- **Objeto (Instancia):** Es la "casa" construida a partir de ese plano. Cuando en tu test escribes `new PaginaLogin()`, estás instanciando un objeto real en la memoria de la computadora que usarás para interactuar con el navegador.

```
// El Plano (Clase)
public class Usuario {
    String nombre; // Atributo

    public void saludar() { // Método
        System.out.println("Hola, soy " + nombre);
    }
}

// La Construcción (Instanciación)
public class Prueba {
    public static void main(String[] args) {
        Usuario user1 = new Usuario(); // user1 es un Objeto
        user1.nombre = "Admin";
        user1.saludar();
    }
}
```

Modificadores de Acceso (`public` , `private` , `protected`)

En automatización, la **encapsulación** es vital.

- `public` : Cualquier clase del proyecto puede verlo. Los métodos (acciones) de tus Page Objects como `iniciarSesion()` siempre deben ser `public` para que tus *Step Definitions* o Tests puedan usarlos.
- `private` : Solo la propia clase puede verlo. **Tus localizadores (XPath, ID) en un Page Object SIEMPRE deben ser `private`**. Ningún test debe interactuar directamente con un XPath; el test debe llamar al método público, y el método público usa el XPath privado.
- `protected` : Solo la propia clase y las clases que "heredan" de ella pueden verlo.

2. Herencia y la palabra `super`

La herencia (palabra clave `extends`) permite que una clase "hija" herede todos los métodos y atributos de una clase "padre". Es la base del **Page Object Model**.

El Problema

Todas tus páginas web necesitan inicializar `WebDriverWait`, necesitan interactuar con el mouse, etc. Escribir esa lógica de espera en las 50 páginas de tu aplicación sería una pesadilla.

La Solución

Creas una clase padre `BasePage` y haces que las 50 páginas hereden de ella.

```
// CLASE PADRE
public class BasePage {
    protected WebDriver driver; // protected para que las hijas lo vean

    // Constructor
    public BasePage() {
        this.driver = DriverManager.getDriver();
    }

    public void clickGenial(By locator) {
        // Lógica súper compleja de esperas antes de hacer clic...
        driver.findElement(locator).click();
    }
}

// CLASE HIJA
public class PaginaCarrito extends BasePage {
    private By btnPagar = By.id("pagar");

    // Al heredar, automáticamente heredas el constructor de BasePage
    // (A menos que declares uno y uses la palabra 'super()')

    public void realizarPago() {
        // clickGenial() no está escrito en PaginaCarrito, ¡pero lo podemos usar gracias a la
        herencia!
        clickGenial(btnPagar);
    }
}
```

3. Estructuras de Control y Bucles

Iterar es el pan de cada día en QA, especialmente al interactuar con Tablas o Grillas web.

Bucle `for` Clásico

Útil cuando necesitas llevar una cuenta numérica o interactuar con elementos mediante su índice (`[i]`).

```
// Recorriendo filas de una tabla
List<WebElement> filas = driver.findElements(By.cssSelector("table tr"));
for (int i = 0; i < filas.size(); i++) {
```

```
        System.out.println("Fila " + i + ": " + filas.get(i).getText());
    }
}
```

Bucle `for-each` (Recomendado)

Más limpio y moderno cuando no te importa el índice numérico, solo quieres los elementos.

```
List<WebElement> enlaces = driver.findElements(By.tagName("a"));
for (WebElement link : enlaces) {
    if (link.getText().contains("Borrar")) {
        link.click();
        break; // Detiene el bucle inmediatamente
    }
}
```

4. Colecciones de Datos (`List` vs `Set`)

Cuando le pides a Selenium múltiples elementos (usando `findElements`), te devuelve una Colección de tipo `List` .

- **`List` (Listas):** Ordenadas y permiten duplicados. Cada elemento tiene un índice (0, 1, 2...).
 - *Uso QA:* `List<WebElement>` para extraer todos los resultados de una búsqueda.
- **`Set` (Conjuntos):** Desordenados y **no permiten duplicados**.
 - *Uso QA:* Crítico para manejar Múltiples Pestañas. `driver.getWindowHandles()` devuelve un `Set<String>` con los identificadores únicos de cada ventana abierta. Como cada pestaña es única, un `Set` garantiza que no haya IDs duplicados.

5. Excepciones: `try-catch-finally`

En Selenium, los elementos a veces no cargan, las conexiones fallan, o el diseño de la página cambia. Las Excepciones son "errores gritando por ayuda".

Si no capturas una excepción, tu suite de pruebas "explota" y se detiene la ejecución.

```
public void clicSeguro(By locator) {
    try {
        // INTENTA hacer esto
        driver.findElement(locator).click();

    } catch (ElementClickInterceptedException e) {
        // SI FALLA por estar interceptado, haz esto (Plan B)
        System.out.println("El elemento está oculto. Usando JavaScriptExecutor...");
        JavascriptExecutor js = (JavascriptExecutor) driver;
        js.executeScript("arguments[0].click();", driver.findElement(locator));

    } finally {
        // Esto se ejecuta SIEMPRE, haya fallado o no.
        // Útil para cerrar conexiones a Bases de Datos o liberar memoria.
        System.out.println("Terminando intento de clic.");
    }
}
```

```
}  
}
```

6. Variables Estáticas vs De Instancia (El peligro del `static WebDriver`)

Una variable normal (de Instancia) le pertenece **exclusivamente al objeto** que fue creado. Una variable `static` le pertenece **a la Clase**, y por tanto es compartida universalmente en todo el programa.

Por qué NO usar `public static WebDriver driver;`

Si declaras tu navegador como estático:

1. Escribes la prueba A y abre Chrome.
2. Comienza la prueba B en paralelo y abre otro Chrome. ¡Ups! Sobrescribió la variable compartida de la prueba A.
3. La prueba A intenta hacer clic, pero interactúa con la pestaña de la prueba B.
4. Todo colapsa.

La Solución Senior: `ThreadLocal<WebDriver>`

Como viste en el `DriverManager.java` de este framework, usamos `ThreadLocal` . Es una variable especial de Java que le dice a la máquina virtual: *"Esta variable pertenece a toda la clase, PERO cada hilo de ejecución (Thread) tendrá una copia secreta e independiente de ella"*. Esto permite correr 50 pruebas en paralelo de forma segura.

Curso Intensivo Paso a Paso: Selenium, Java y Cucumber BDD

Bienvenido al curso definitivo de automatización de pruebas integradas. Este manual está diseñado como un libro de texto guiado. Al finalizar, serás capaz de construir arquitecturas de QA sólidas y escalables.

 **Pre-requisito:** Antes de dominar la arquitectura, debes saber cómo interactuar con el HTML. Por favor, lee de forma obligatoria el [Manual de Localizadores](#) adjunto en este framework.

Módulo 1: Fundamentos y Configuración Inicial (Paso a Paso)

1.1 Entendiendo Selenium WebDriver

Selenium no "testea" nativamente. Selenium simplemente provee una API (Application Programming Interface) que se comunica directamente con el navegador utilizando el **Protocolo W3C WebDriver**. Cuando en Java escribimos `.click()`, Selenium traduce eso en un comando HTTP estandarizado que el navegador entiende y ejecuta.

1.2 Paso 1: Configurar el proyecto (Maven)

Maven es el corazón de la gestión en Java. En lugar de descargar librerías manualmente, usamos el archivo `pom.xml`.

¿Cómo debe lucir un `pom.xml` básico?

```
<dependencies>
  <!-- El núcleo de Selenium -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.23.1</version>
  </dependency>
  <!-- Motor de Pruebas -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.11.0</version>
    <scope>test</scope>
  </dependency>
  <!-- Gestor Automático de Navegadores -->
  <dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>6.3.3</version>
  </dependency>
</dependencies>
```

1.3 Paso 2: Hola Mundo en Selenium (La forma rústica)

Antes de construir arquitecturas complejas, veamos cómo se abre una página de forma directa:

```
import io.github.bonigarcia.wdm.WebDriverManager;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
```

```

public class HolaMundoSelenium {
    public static void main(String[] args) {
        // 1. Descarga el driver adecuado para tu versión de Chrome
        WebDriverManager.chromedriver().setup();

        // 2. Levanta el navegador (Sesión)
        WebDriver driver = new ChromeDriver();

        // 3. Maximiza la ventana y navega
        driver.manage().window().maximize();
        driver.get("https://www.google.com");

        // 4. Imprime el título para verificar
        System.out.println("El título es: " + driver.getTitle());

        // 5. Destruye la sesión (Crítico para no consumir RAM infinita)
        driver.quit();
    }
}

```

Módulo 2: Cucumber y el Diseño Basado en Comportamientos (BDD)

BDD acorta la brecha entre desarrolladores, QA y negocio (Product Owners). Las pruebas se escriben en lenguaje humano (Gherkin).

2.1 Escribiendo Escenarios (Archivos `.feature`)

```

Feature: Autenticación de Usuario
  Como cliente registrado
  Quiero iniciar sesión
  Para acceder a mi panel privado

  Scenario: Iniciar sesión exitosamente
    Given que navego a la página de login
    When ingreso el usuario "admin@test.com" y la clave "12345"
    Then el panel de control es visible

```

2.2 Paso a Paso: Mapeando Gherkin a Java (Step Definitions)

Cada oración del archivo `.feature` debe coincidir con un método en Java.

```

public class LoginSteps {

    // El texto dentro de @Given debe coincidir EXACTAMENTE con el .feature
    @Given("que navego a la página de login")
    public void navegarAlLogin() {
        // Aquí interactuaremos con el navegador más adelante
        System.out.println("Navegando...");
    }

    // Pasando parámetros desde Gherkin (usando {string})

```

```

@When("ingreso el usuario {string} y la clave {string}")
public void ingresarCredenciales(String user, String password) {
    System.out.println("Login con: " + user + " y " + password);
}
}

```

Módulo 3: Profundización Absoluta en Page Object Model (POM)

3.1 El problema del Código Espagueti (Lo que NO debes hacer)

Imagina tener 50 pruebas que inician sesión así:

```

// MAL EJEMPLO
driver.findElement(By.id("user_email")).sendKeys("test@test.com");
driver.findElement(By.id("user_password")).sendKeys("123");
driver.findElement(By.cssSelector("button[type='submit']")).click();

```

Si el desarrollador cambia el ID `user_email` a `email_address`, ¡tendrás que corregir 50 archivos!

3.2 Paso a Paso: Refactorizando al Patrón POM

El POM centraliza la página en un solo archivo.

Paso 1: Crear la clase de la página (La Vista)

```

public class PaginaLogin {
    // 1. Encapsulamiento: Los localizadores son privados.
    private By emailInput = By.id("user_email");
    private By passwordInput = By.id("user_password");
    private By submitButton = By.cssSelector("button[type='submit']");

    // 2. WebDriver inyectado
    private WebDriver driver;

    public PaginaLogin(WebDriver driver) {
        this.driver = driver;
    }

    // 3. Acciones de negocio (Públicas)
    public void ingresarCredenciales(String email, String pass) {
        driver.findElement(emailInput).sendKeys(email);
        driver.findElement(passwordInput).sendKeys(pass);
        driver.findElement(submitButton).click();
    }
}

```

Paso 2: Herencia (El nivel Senior) En nuestro framework `FSeleniumIA`, no queremos inyectar el `WebDriver` y llamar a `driver.findElement` mil veces. Creamos una `BasePage` que hace el trabajo sucio.

```
// Así luce en FSeleniumIA (Elegante y Limpio)
public class PaginaLogin extends BasePage {
    private By emailInput = By.id("user_email");

    public void ingresarEmail(String email) {
        // write() es heredado de BasePage, ya incluye esperas explícitas automáticas.
        write(emailInput, email);
    }
}
```

Módulo 4: TestNG y Aserciones Avanzadas

4.1 Aserciones (Validando resultados)

Una prueba no es una prueba si no verifica algo. TestNG nos da aserciones poderosas.

```
// Hard Assert: Si esto falla, la prueba se aborta INMEDIATAMENTE.
Assert.assertEquals(textoObtenido, "Bienvenido", "El texto no coincide.");
Assert.assertTrue(elemento.isDisplayed(), "El botón debería estar visible.");

// Soft Assert: Si falla, la prueba CONTINÚA y reporta todas las fallas al final.
SoftAssert softAssert = new SoftAssert();
softAssert.assertEquals(color, "Rojo");
softAssert.assertTrue(estaVisible);
softAssert.assertAll(); // Crítico: Si no llamas esto, la prueba siempre pasa.
```

4.2 Ejecución Parametrizada (@DataProvider)

Permite correr un test de TestNG múltiples veces con datos distintos.

```
@DataProvider(name = "credenciales")
public Object[][] datosLogin() {
    return new Object[][] {
        {"admin@test.com", "1234"},
        {"user@test.com", "pass123"},
        {"bloqueado@test.com", "0000"}
    };
}

// El test se ejecutará 3 veces automáticamente
@Test(dataProvider = "credenciales")
public void testLogin(String user, String pass) {
    PaginaLogin login = new PaginaLogin();
    login.iniciarSesion(user, pass);
}
```


Módulo 5: Interacciones Complejas en Selenium

5.1 Formularios: La Clase `Select`

Para interactuar con listas desplegables (Dropdowns) nativas `<select>`.

```
WebElement dropdown = driver.findElement(By.id("paises"));
Select selectPais = new Select(dropdown);

// 3 formas de seleccionar:
selectPais.selectByVisibleText("Argentina"); // Lo que ve el usuario (Mejor)
selectPais.selectByValue("ARG");             // Por el atributo 'value' HTML
selectPais.selectByIndex(2);                 // La tercera opción (Poco seguro)
```

5.2 Estrategias de Espera (Sincronización)

 **ERROR COMÚN:** Usar `Thread.sleep(5000)`. Congela todo y hace la ejecución muy lenta.

Opción A: Implicit Wait (Global)

```
// Espera HASTA 10 segundos intentando buscar elementos.
// Si aparece en el segundo 2, avanza inmediatamente.
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

Opción B: Explicit Wait (Específico) La mejor práctica absoluta. Espera por una condición.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
// Esperar hasta que un botón invisible se vuelva clickable
WebElement btn = wait.until(ExpectedConditions.elementToBeClickable(By.id("submit")));
btn.click();
```

5.3 Simulación de Mouse: Clase `Actions`

Para arrastrar y soltar (Drag and Drop), mover el mouse por encima (Hover), etc.

```
Actions actions = new Actions(driver);
WebElement arrastrable = driver.findElement(By.id("caja1"));
WebElement destino = driver.findElement(By.id("caja2"));

// Arrastrar y Soltar
actions.dragAndDrop(arrastrable, destino).perform();

// Mover el mouse por encima de un menú para que se despliegue
WebElement menu = driver.findElement(By.id("menu-desplegable"));
actions.moveToElement(menu).perform();
```

Módulo 6: Estándares Arquitectónicos de un QA Senior

1. **Gestión de Hilos (Thread-Safety):** Si corres pruebas en paralelo, el hilo 1 y el hilo 2 chocarán si comparten una variable `static WebDriver driver`. Usamos `ThreadLocal<WebDriver>` en nuestra clase `DriverManager` para que cada hilo tenga su propia copia del navegador, aislada de las demás.
2. **Abstracción Total de URLs:** No debe haber ni un solo `http://...` en tu código Java. Se lee todo dinámicamente usando una clase custom como `ConfigReader.java` que extrae los datos de `config.properties`.
3. **Manejo de Errores Silencioso:** En los *Hooks* de Cucumber, si un escenario falla (`scenario.isFailed()`), inyectamos un bloque `try-catch` que toma un *Screenshot* nativo en bytes y lo pega en el reporte final automáticamente, sin detener abruptamente el flujo de limpieza.

Apéndices del Framework (Crecimiento Futuro)

Apéndice A: Data-Driven Avanzado

Para escenarios donde el `@DataProvider` no alcanza, el framework incluye la carpeta `dataloader/`. Usando la librería **Apache POI** para Excel o **OpenCSV** para archivos de texto, puedes cargar miles de filas para pruebas transaccionales masivas, inyectando esos valores a tus variables durante la inicialización de la prueba.

Apéndice B: CI/CD y Headless Testing

Cuando se integran las pruebas en GitHub Actions o Jenkins, la máquina virtual no tiene pantalla. El navegador debe iniciar en "Headless Mode" (invisible).

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless", "--window-size=1920,1080"); // Window size crítico para
responsividad
driver = new ChromeDriver(options);
```

Apéndice C: Estrategias de Debugging en IDE

Cuando recibes una excepción inentendible como `StaleElementReferenceException`:

1. El DOM se recargó y el `WebElement` que guardaste en una variable quedó "viejo".
2. **Solución:** Pon un *Breakpoint* (Punto de interrupción rojo) en IntelliJ, ejecuta en modo *Debug*, y usa *Evaluate Expression* para forzar una nueva búsqueda de ese elemento en vivo.

Apéndice D: Integración de API (RestAssured)

Un framework Senior no automatiza precondiciones a través de la Interfaz Gráfica de Usuario (GUI) si es lento.

- **Caso:** Necesito probar cómo borrar un post.
- **Malo:** Selenium navega a "Crear post", llena formularios, y luego va a borrarlo. (Tarda 1 minuto).
- **Senior:** Usa RestAssured (HTTP Client Java) en el `@Before` para enviar un JSON por POST, creando el artículo en la Base de Datos en milisegundos. Luego, abre Selenium directo en la URL del post y hace clic en Borrar.

Apéndice E: Escalabilidad con Selenium Grid

Cuando 100 pruebas demoran demasiado en correr en una PC, usamos Grid. Levantas un Hub central y Nodos satélites (usualmente con contenedores Docker). En el código, sustituyes `new ChromeDriver()` por `new RemoteWebDriver(new URL("http://hub:4444"), options)`. Esto deriva las pruebas a las nubes.

Apéndice F: El Patrón Factory

Para escalar el soporte de navegadores (Chrome, Edge, Firefox, Safari, Appium Mobile), en lugar de tener un `switch-case` interminable, se aplica el Patrón de Diseño Factory. Se crea una interfaz `BrowserFactory` y múltiples implementaciones (ej: `ChromeManager` , `FirefoxManager`). El framework decide en tiempo de ejecución qué manejador instanciar basándose en las variables de entorno.

Manual de Estrategias de Localizadores en Selenium

Este manual detallado cubre todas las estrategias de localización disponibles en Selenium WebDriver, con un enfoque en aplicaciones prácticas y código en Java. En la automatización de pruebas, la elección correcta de un localizador es crucial para la estabilidad y el mantenimiento a largo plazo de tus scripts.

1. Introducción a los Localizadores

Un "Locator" o Localizador es un método que le indica a Selenium cómo encontrar un elemento web específico dentro del DOM (Document Object Model) de una página HTML. WebDriver utiliza la interfaz `By` para definir la estrategia de búsqueda.

Regla de Oro: Un buen localizador debe ser único, descriptivo y poco propenso a cambiar cuando la interfaz gráfica reciba actualizaciones menores.

2. Estrategias Básicas de Localización

2.1 Por ID (`By.id`)

Es la forma más rápida y segura de ubicar un elemento, ya que el estándar HTML indica que un ID debe ser único dentro de la página.

Ejemplo HTML:

```
<input type="text" id="username_field" name="usuario" class="input-form">
```

Uso en Java:

```
By locatorId = By.id("username_field");
driver.findElement(locatorId).sendKeys("admin");
```

2.2 Por Name (`By.name`)

Útil cuando los elementos (especialmente inputs de formularios) tienen un atributo `name`. A diferencia del ID, el `name` puede no ser único (ej. radio buttons). Selenium devolverá el primer elemento que coincida.

Uso en Java:

```
By locatorName = By.name("usuario");
driver.findElement(locatorName).sendKeys("admin");
```

2.3 Por Class Name (`By.className`)

Busca elementos basándose en el valor del atributo `class`. No es recomendable si la clase es muy genérica (ej. `btn-primary`), ya que devolverá el primer botón que encuentre, que podría no ser el que buscas. **Nota:** No soporta clases múltiples separadas por espacios.

Ejemplo HTML:

```
<button class="btn btn-success submit-btn">Enviar</button>
```

Uso en Java:

```
By locatorClass = By.className("submit-btn");  
driver.findElement(locatorClass).click();
```

2.4 Por Tag Name (`By.tagName`)

Localiza elementos por el nombre de la etiqueta HTML (`a` , `input` , `button` , `div`). Es muy útil para extraer colecciones de elementos.

Uso en Java (Extraer todos los enlaces):

```
List<WebElement> enlaces = driver.findElements(By.tagName("a"));  
System.out.println("Total de enlaces en la página: " + enlaces.size());
```

2.5 Por Link Text (`By.linkText`) y Partial Link Text (`By.partialLinkText`)

Exclusivos para etiquetas de anclaje `<a>` . Buscan por el texto visible del enlace.

- `linkText` busca una coincidencia exacta.
- `partialLinkText` busca una subcadena (muy útil si el texto del enlace cambia dinámicamente).

Ejemplo HTML:

```
<a href="/login">Haz clic aquí para Iniciar Sesión</a>
```

Uso en Java:

```
By exacto = By.linkText("Haz clic aquí para Iniciar Sesión");  
By parcial = By.partialLinkText("Iniciar Sesión");
```

3. Selectores Avanzados: CSS Selectors (`By.cssSelector`)

Los selectores CSS son extremadamente rápidos en la ejecución nativa del navegador. Su sintaxis es idéntica a la que usan los desarrolladores frontend en sus hojas de estilo.

Sintaxis Básica CSS

- **ID:** Se usa `#` -> `#username_field`
- **Class:** Se usa `.` -> `.submit-btn`
- **Tag:** Se usa el nombre directo -> `input`

Sintaxis por Atributos

```
// Busca un input cuyo atributo 'placeholder' sea 'Email'
By cssAtributo = By.cssSelector("input[placeholder='Email']");
```

Comodines en CSS (Expresiones Regulares Básicas)

- **Empieza con (^=)**: `input[id^='user_']` (Encontrará `user_123` , `user_name`)
- **Termina con (\$=)**: `input[id$='_field']` (Encontrará `username_field` , `password_field`)
- **Contiene (*=)**: `button[class*='success']` (Encontrará `btn-success-login`)

Combinadores CSS

- **Hijo Directo (>)**: `div.panel > p` (Párrafos que son hijos inmediatos del div)
- **Descendiente (espacio)**: `div.panel p` (Cualquier párrafo dentro del div, sin importar la profundidad)
- **Hermanos (+ o ~)**: `h1 + p` (El primer párrafo inmediatamente después de h1)

4. Selectores Avanzados: XPath (By.xpath)

XPath (XML Path Language) permite navegar por toda la estructura del DOM en cualquier dirección (hacia arriba o hacia abajo). Es la herramienta más poderosa cuando no tienes IDs ni clases estables.

XPath Absoluto vs Relativo

- **Absoluto**: Empieza desde la raíz `/html/body/div[1]/form/input` . **Nunca lo uses**, si cambia un solo div en el HTML, el test fallará.
- **Relativo**: Empieza en cualquier nodo de la página mediante `//` . Es el estándar de la industria.

Sintaxis Básica XPath Relativo

Formato general: `//etiqueta[@atributo='valor']`

```
By xpathBasico = By.xpath("//input[@name='usuario']");
By xpathMultiple = By.xpath("//input[@name='usuario' and @type='text']");
```

Funciones de Texto en XPath

- `text()` : Coincidencia exacta del texto visible del nodo.

```
By xpathText = By.xpath("//button[text()='Enviar Formulario']");
```

- `contains()` : Muy potente para textos o atributos dinámicos (que cambian parcialmente).

```
// Busca un botón cuyo texto contenga "Enviar"
By xpathContainsTxt = By.xpath("//button[contains(text(), 'Enviar')]");
```

```
// Busca un div cuya clase contenga "active"
By xpathContainsAttr = By.xpath("//div[contains(@class, 'active')]");
```

- **starts-with()** : Para elementos cuyos IDs o clases se generan dinámicamente, pero su inicio es constante.

```
By xpathStartsWith = By.xpath("//input[starts-with(@id, 'session_')]");
```

Navegación por Ejes XPath (Axes)

Los ejes permiten buscar nodos relativos a otro nodo conocido.

1. Localizar al Padre (**parent** o **..**)

```
By xpathPadre = By.xpath("//input[@id='username']/parent::div");
// Alternativa corta: "//input[@id='username']/.."
```

2. Localizar Hermanos Posteriores (**following-sibling**)

```
// Útil en tablas para encontrar el valor al lado de una etiqueta
By xpathHermano = By.xpath("//td[text()='Total']/following-sibling::td");
```

3. Localizar Ascendientes Lejanos (**ancestor**)

```
// Busca la tabla (ancestor) que contiene la celda con el texto "Dato 1"
By xpathAncestro = By.xpath("//td[text()='Dato 1']/ancestor::table");
```

5. CSS Selector vs XPath: ¿Cuál elegir?

Característica	CSS Selector	XPath
Rendimiento / Velocidad	Ligeramente más rápido (optimizado por navegadores).	Ligeramente más lento.
Legibilidad	Corto, limpio y muy legible.	Tiende a ser más verboso y complejo.
Flexibilidad	Excelente, pero solo navega hacia abajo (DOM).	Superior. Permite navegar hacia arriba (Padres/Ancestros).
Búsqueda por Texto	No soportado nativamente (a menos que dependa de IDs/clases).	Altamente soportado (text() , contains()).

Veredicto Senior: Prioriza siempre **ID** o **Name**. Si no están disponibles, usa **CSS Selectors** por su velocidad y legibilidad. Usa **XPath** cuando necesites búsquedas complejas basadas en texto o navegar hacia elementos padre/hermanos.

6. Mejores Prácticas de QA (Framework FSeleniumIA)

1. **Atributos de Datos Personalizados** (`data-testid` , `data-cy`): Lo ideal es pedir a los desarrolladores que agreguen selectores exclusivos para pruebas.

```
By locatorTestId = By.cssSelector("[data-testid='login-button']"); // A prueba de balas
```

2. **No usar selectores autogenerados del navegador:** No copies el selector directamente del menú "Copy Selector" de Chrome DevTools si se ve así: `#root > div > div:nth-child(2) > form > input` . Son extremadamente frágiles.
3. **Evita la indexación estática en XPath:** Trata de evitar `//table/tr[4]/td[2]` . Si se agrega una fila nueva arriba, tu prueba se romperá. Usa referencias relativas al contenido, ej: `//td[text()='Juan']/following-sibling::td` .

Explicaciones, Tips y Buenas Prácticas

Este documento centraliza los conocimientos clave para entender cómo funciona la arquitectura de pruebas híbridas en Java y cómo sacarle el máximo provecho al diseño.

1. El Flujo de Ejecución de Cucumber (BDD)

Cuando ejecutas una prueba orientada a comportamiento (BDD) mediante el archivo `TestRunner`, el framework realiza el siguiente flujo:

1. **Feature File (`.feature`)**: Cucumber lee el escenario escrito en Gherkin (texto en lenguaje natural).
 2. **Hooks (`Hooks.java`)**: Se dispara el método `@Before`. Aquí el `DriverManager` arranca un nuevo navegador exclusivo para este escenario de manera totalmente aislada.
 3. **Step Definitions (`steps/*.java`)**: Cucumber mapea la frase Gherkin con su método en Java. Ejemplo: `Given I navigate to...` ejecuta la función correspondiente en la clase Steps.
 4. **Page Objects (`pages/*.java`)**: Los métodos en los Steps interactúan con las páginas. Las páginas encapsulan las interacciones directas de Selenium.
 5. **Hooks (Fin)**: Terminan los pasos, y se dispara el método `@After`. Si el test falló, saca una captura de pantalla. Finalmente, destruye la sesión del navegador.
-

2. El Patrón Page Object Model (POM) con Herencia

El POM es un patrón de diseño que evita la duplicación de código y permite que, si la interfaz de usuario cambia, solo tengamos que actualizar un único lugar en nuestro código.

En `FSeleniumIA` implementamos el patrón con **Herencia**:

- Tenemos una clase `BasePage.java` que posee el WebDriver y métodos comunes (como `clickElement`, `write`, esperas implícitas, interacciones con Dropdowns).
 - Cada nueva página (ej: `PaginaRegistro.java`) debe **heredar** (`extends BasePage`) de esta clase padre.
 - *Beneficio didáctico*: No tienes que re-escribir toda la lógica de esperas ni interactuar directo con el WebDriver en cada página. Solo te concentras en definir los localizadores (XPaths, CSS Selectors) y acciones de la vista.
-

3. Manejo de Esperas: La Regla de Oro

Selenium ofrece esperas para sincronizar el código con la velocidad del navegador.

✗ Tip Crítico: NUNCA utilices `Thread.sleep(5000)`; Esta función congela el hilo de Java estáticamente por 5 segundos enteros, incluso si el elemento ya cargó en el segundo 1. Esto infla drásticamente el tiempo total de tu suite de pruebas.

✓ **Uso Correcto (Esperas Explícitas)**: El `BasePage` de este framework ya utiliza `WebDriverWait`. Cuando usas un método como `clickElement`, internamente el `BasePage` evalúa si el elemento está presente y clickeable usando `ExpectedConditions`. En cuanto aparece, avanza; si no, espera hasta un `timeout` definido.

4. Consejos de Depuración en IntelliJ

Si un test de automatización está fallando y no sabes por qué, en lugar de agregar "prints", usa el Debugger:

1. Pon un punto de interrupción (red dot) en la línea de código donde crees que está el error (ej: en el Step o en la Acción del Page Object).
2. Haz clic derecho sobre la prueba y en lugar de "Run", elige **"Debug"**.
3. El navegador se abrirá, y la ejecución se pausará justo en tu línea de código.
4. Puedes utilizar la consola "Evaluate Expression" (Alt + F8) en IntelliJ para probar localizadores XPath en tiempo real sobre el navegador pausado.